

Alias Inteligentes v3.0: Enrutamiento al Entorno Virtual (Runtime vs Buildtime)

Cuadernillo | Vol. 1

mercedev.es — 2026-05-04

El Desafío (Síntoma)

A medida que el ecosistema Merci crecía integrando librerías de compilación locales pesadas (como `markdown` para el motor SSG, `Pillow` para optimización de imágenes o `WeasyPrint` para generación de PDFs), la fricción operativa aumentó drásticamente.

El síntoma principal era que al abrir una nueva terminal y ejecutar `merci total` o `merci publish`, el sistema fallaba con errores de "módulo no encontrado" (*ModuleNotFoundError*). El motivo radicaba en el olvido frecuente de ejecutar `source .venv/bin/activate` para ingresar al entorno virtual de Python. La frustración llegó al extremo de considerar abandonar el estándar Markdown y programar un parseador propio de `.txt` crudo con expresiones regulares, solo para evadir el uso de librerías.

La Maniobra (Lógica)

En lugar de destruir la arquitectura documental (que se apoya fuertemente en el formato Markdown) y generar deuda técnica masiva reinventando la rueda, se aplicó una solución nativa de sistemas POSIX.

Se actualizó la función enrutadora `merci()` en el archivo `~/.zshrc` (escalando de la v2.0 a la v3.0) para que invoque explícitamente el binario de Python encapsulado dentro del entorno virtual, sustituyendo la llamada genérica al comando `python3` global.

```
# Motor Merci - Ejecutor Inteligente v3.0
merci() {
    if [ -f "scripts/merci/merci-$1.py" ]; then
        # QUÉ HACE: Llama explícitamente al binario de Python dentro
        de .venv.
        # POR QUÉ: Evita tener que teclear 'source .venv/bin/activate'
        en cada sesión.
```

```
.venv/bin/python "scripts/merci/merci-$1.py" "${@:2}"
else
    echo "🛡️ [Merci Error] No estás en la raíz de un proyecto Merci
o el comando '$1' no existe."
fi
}
```

Al guardar el archivo y ejecutar `source ~/.zshrc`, el sistema operativo resuelve de forma automática las librerías instaladas en ese entorno aislado sin necesidad de que la sesión de la terminal se encuentre formalmente "activada".

El Aprendizaje (Deuda Técnica)

Esta crisis operativa sirvió para consolidar una barrera arquitectónica vital en nuestra Gobernanza:

1. **Runtime (Entorno de Ejecución):** Aquí rige la ley innegociable de **0 dependencias**. En el navegador del usuario solo viaja HTML5, CSS (SASS compilado) y Vanilla JS. Cero frameworks, cero librerías, cero latencia.
2. **Buildtime (Entorno de Compilación/Pipeline):** Aquí **sí** usamos herramientas. Es nuestro entorno local DevSecOps. Utilizar herramientas estables como *WeasyPrint* o *Pillow* localmente no vulnera la filosofía del proyecto, porque estas librerías no viajan a producción. Simplemente actúan como brazos robóticos ensambladores en nuestra fábrica de código.

Un ecosistema de desarrollo profesional debe priorizar siempre la *Developer Experience* (DX). Erradicar una fricción de la terminal dominando las rutas nativas de Linux es infinitamente superior a degradar la calidad de los datos y el formato de los contenidos para saltarse el uso de entornos virtuales.